API Pagination: Offset vs Cursor

and why deep pages are slow

Watch the 4-min video

In one line: offset pagination makes the database read and discard every skipped row, so deep pages get slow and shift when data changes; cursor (keyset) pagination remembers where you stopped and seeks there through an index, so page 500 costs the same as page 1.

1 Why pagination design matters

An infinite-scroll feed has **50 million posts**. Nobody downloads them all, so the API returns one page at a time. How you define the page boundary decides two things: whether page 500 loads in **5 ms or 5 seconds**, and whether users **see duplicates or miss posts** while scrolling.

2 Offset and limit: simple, with two big problems

“Give me 20 items, skipping the first 40”:

```
SELECT * FROM posts
ORDER BY created_at DESC
LIMIT 20 OFFSET 40;          -- page 3
```

It is simple, and it lets you **jump straight to page 7** (offset = (page − 1) × size), which is great for admin tables with page numbers. But:

- **Deep offsets are slow.** To return rows 1,000,000 to 1,000,020, the database still walks past the first million rows (in the index or the table) and throws them away. The cost grows with the page number.
- **It is unstable when data changes.** Positions are counted fresh on every request, so an insert or delete between two requests shifts every row.

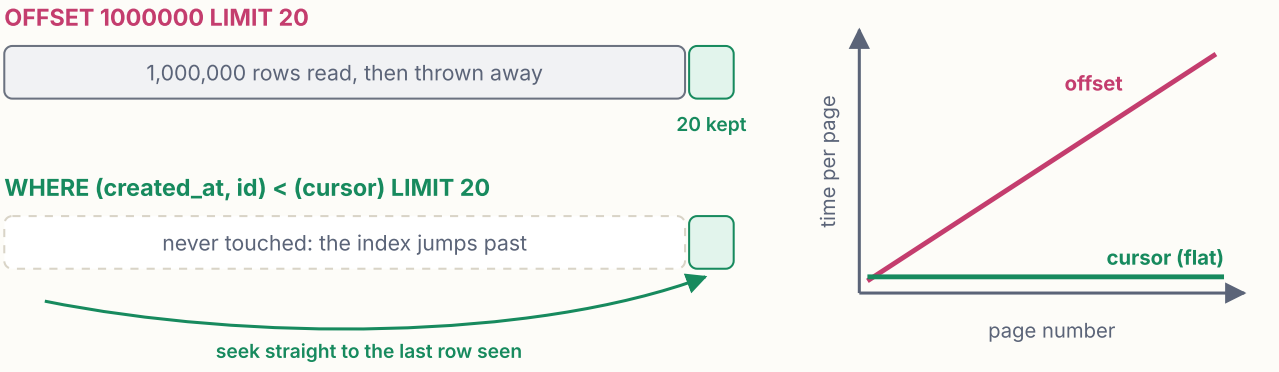


Figure 1. Offset cost grows linearly with depth because skipped rows are still read. A keyset query seeks to the boundary through the index, so cost stays flat.



Figure 2. A new post arrives between page 1 and page 2. Offset 3 now points at post 98, which the user already saw. A delete does the opposite: rows shift up and one post is silently skipped.

3 Cursor (keyset) pagination

Instead of saying **how many rows to skip**, the client says **where it stopped**: the sort values of the last row it saw.

```
SELECT * FROM posts
WHERE (created_at, id) < ($last_created_at, $last_id)
ORDER BY created_at DESC, id DESC
LIMIT 20;

-- needs: CREATE INDEX ON posts (created_at DESC, id DESC);
```

With a composite index on `(created_at, id)`, the database jumps straight to that point in the index and reads exactly 20 entries. Page 500 costs the same as page 1. New posts at the top don't move the boundary, so there are no duplicates; deleted posts simply aren't returned, so nothing is skipped.

The row comparison `(a, b) < (x, y)` works in PostgreSQL. Where it isn't supported or doesn't use the index well, write it out: `created_at < $t OR (created_at = $t AND id < $id)`.

The tie-breaker is not optional

Ordering must be **stable and unique**. Many posts can share the same timestamp (a bulk import can write thousands in one millisecond). If you sort only by `created_at` and a page ends in the middle of a group of equal timestamps, `created_at < t` skips the rest of that group and `created_at <= t` repeats it. Adding the unique `id` as the last sort column gives every row an exact position.

4 The API shape: an opaque cursor

Don't expose raw database values as parameters. Encode the last `created_at` and `id` into an **opaque cursor**, for example Base64 of a small JSON object, ideally **signed** (an HMAC) so clients can't tamper with it or craft cursors that probe other data. The server is free to change what's inside later.

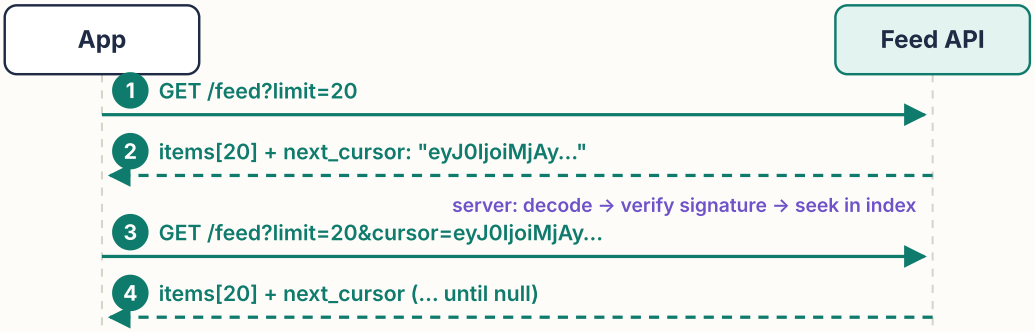


Figure 3. The response carries the items and a next_cursor. The client sends it back unchanged. When next_cursor is null, there are no more pages. The client treats the cursor as an opaque string.

```
{
  "items": [ {"id": 9182, "created_at": "2026-11-04T09:12:44Z", ...}, ... ],
  "next_cursor": "eyJ0IjoiMjAyNi0xMS0wNFQwOToxMjo0NFoiLCJpZCI6OTE4Mn0.sig",
  "has_more": true
}
```

WORKED EXAMPLE

Page size 20, user scrolls to page 500: offset = 499 × 20 = **9,980**. With offset, the database reads about 10,000 index entries to return 20; at page 50,000 it reads about **1,000,000**, and if each read is ~5 µs that's ~5 s per page. With a cursor, every page reads about **20 entries** plus one index descent (a few page reads in a B-tree of height 3–4), well under 5 ms at any depth.

Fetching `LIMIT 21` and returning 20 is a cheap way to know `has_more` without counting the table.

5 Trade-offs and limits

	Offset / limit	Cursor / keyset
Deep page cost	Grows with the offset	Constant (with the right index)
Inserts / deletes while paging	Duplicates or skipped rows	Stable
Jump to page 50	Yes	No: next (and maybe previous) only
Total count	Often shown, but `COUNT(*)` on 50M rows is expensive	Usually `has_more` instead
Change sort order	Any time	Starts a new cursor (tied to one order)
Best for	Small admin tables with page numbers	Feeds, timelines, APIs, data exports

Either way, **cap the page size**: a default of 20 and a maximum of 100, so `limit=1000000` can't turn one request into a full table dump.

COMMON MISTAKES

- Sorting by a non-unique column (only `created_at`) with no tie-breaker: rows repeat or vanish at page edges.
- No index matching the sort, so the "seek" becomes a sort of the whole table on every page.
- Exposing raw values (`?after_id=9182`) that clients start building logic on; you can never change them.
- Returning an exact total count on every page of a huge table.

6 Key terms

Offset pagination: skip N rows, return the next M

Tie-breaker: unique column that makes the order total

Keyset pagination: continue after the last sort key seen

Composite index: index on several columns, e.g. (created_at, id)

Cursor: opaque token encoding that sort key

has_more: flag instead of an exact total

7 Interview questions

Q1. Why is offset pagination slow for deep pages? BASIC

The database must read and discard every skipped row before returning the page, so OFFSET 1,000,000 reads about a million entries to return 20. Cost grows linearly with the page number.

Q2. How does cursor pagination work? BASIC

The client sends an opaque cursor encoding the last sort key, e.g. (created_at, id). The server decodes it, runs WHERE (created_at, id) < (t, id) ORDER BY created_at DESC, id DESC LIMIT 20 on a matching index, and returns the page plus a new next_cursor (null at the end).

Q3. Why add id to the ORDER BY? BASIC

Timestamps aren't unique. Without a unique tie-breaker, rows with equal created_at at a page boundary are skipped or repeated. created_at + id gives every row an exact position.

Q4. Users report seeing the same post twice while scrolling. Likely cause? SCENARIO

Offset pagination on a feed with new inserts: each new post shifts every row down, so the next offset repeats one. Switch to keyset pagination.

Q5. An admin UI needs page numbers on a 2,000-row table. Cursor or offset? SCENARIO

Offset is fine: 2,000 rows is tiny, users want to jump to page 7 and see a total. Cursors pay off on large, changing data like feeds.

Q6. Page 1 takes 4 ms but page 3,000 takes 3 s. Fix? SCENARIO

It's offset depth: 3,000 × 20 = 60,000 rows skipped. Add a (created_at, id) index and move to cursors; for old deep links, cap the maximum offset (e.g. 10,000) and suggest filters.

Q7. Why make the cursor opaque and signed instead of ?after_id=9182? TRICKY

Opaque lets you change what's inside (add a sort key, switch columns) without breaking clients. Signing stops clients from forging cursors to bypass filters or probe data.

Q8. How do you return a total count for 50M rows? TRICKY

Often you don't: return has_more (fetch 21, return 20). If the UI needs a number, use an approximate count (e.g. planner statistics) or a cached counter, labelled “about 50M”.

Next: SD-C-008 · Filtering, Sorting & Field Selection backendroadmap.com